

你是一名资深 Flutter 工程师，请在当前仓库中实现一个“音程听辨训练”App。

一、项目目标

开发一个可以运行在以下平台的 Flutter 应用：

- Android
- Windows

App 的核心不是普通的随机答题，而是：

1. 播放音程并让用户识别。
2. 记录用户把某个音程误认为哪个音程。
3. 建立用户的音程混淆矩阵。
4. 自动针对最容易混淆的音程安排二选一训练。
5. 错题后提供正确答案与错误答案的交替对比播放。
6. 分别统计不同方向、根音模式和音色下的掌握情况。

请直接完成代码实现，不要只提供架构建议或伪代码。

二、开始开发前

先检查当前项目：

- 阅读 `pubspec.yaml`
- 阅读现有 `lib` 目录结构
- 阅读仓库中的 `README.md`、`AGENTS.md` 或其他工程规范
- 执行 `git status`
- 保留用户已有且与本任务无关的改动
- 优先复用项目已有的路由、状态管理、依赖注入和主题方案
- 不要为了统一风格而大范围重构无关代码

如果当前仓库只是空 Flutter 项目，则按照本文后面的建议结构创建。

新增第三方依赖前：

1. 确认同时支持 Android 和 Windows。
 2. 优先选择维护活跃、跨平台行为稳定的依赖。
 3. 说明引入该依赖的原因。
 4. 不要引入与 MVP 无关的重量级框架。
-

三、技术要求

1. Flutter 与平台

应用必须：

- 支持 Android 触摸操作
- 支持 Windows 鼠标操作
- 支持 Windows 键盘快捷键
- 响应式适配手机和桌面窗口
- 不依赖网络
- 不要求登录
- 所有训练记录保存在本地
- 应用重启后数据仍然存在

Windows 端建议设置合理的最小窗口尺寸，避免内容严重挤压。

2. 推荐架构

如果项目已有固定架构，遵循现有架构。

如果没有，则采用轻量的 Feature-first 分层：

```
lib/  
  app/  
    app.dart  
    router.dart  
    theme.dart  
  
  core/  
    audio/  
    storage/  
    utils/  
    widgets/  
  
  features/  
    home/  
      presentation/  
  
    training/  
      domain/  
      data/  
      presentation/  
  
    report/  
      domain/  
      presentation/
```

```
settings/  
presentation/
```

约束：

- 音程计算、出题算法、统计逻辑放在纯 Dart domain 层
- domain 层不能依赖 Flutter UI
- 音频播放通过抽象接口调用
- 本地存储通过 repository 接口调用
- UI 层不能直接操作文件
- 出题算法必须可以注入随机数种子，以便单元测试
- 不要把全部逻辑堆在一个页面或一个 Cubit 中

状态管理优先使用项目已有方案。

如果没有现成方案，可以使用：

- flutter_bloc 的 Cubit
- 或者结构清晰的 ChangeNotifier

不要同时引入多套状态管理。

四、音程定义

MVP 支持一个八度以内的 13 种音程：

半音数	音程
0	纯一度
1	小二度
2	大二度
3	小三度
4	大三度
5	纯四度
6	增四度 / 减五度
7	纯五度
8	小六度
9	大六度
10	小七度
11	大七度
12	纯八度

请为每个音程定义：

- 唯一 ID
- 中文名称
- 英文简称，例如 `m6`、`M6`
- 半音数
- 默认排序
- 是否可以参与训练
- 可选的简短说明

不要通过字符串比较判断音程，使用枚举或明确的值对象。

五、训练维度

1. 播放方向

支持：

- 上行旋律音程
- 下行旋律音程
- 和声音程，即两个音同时播放
- 随机混合

2. 根音模式

支持三种模式：

固定根音

默认使用 C4。

适合初学者集中辨别不同音程。

有限随机根音

从一个受控范围中选择根音，例如：

- C4
- D4
- E4
- F4
- G4

同时确保目标音不会超出允许音域。

完全随机根音

根音随机，但两个音都必须位于合理音域内。

建议把允许音域限制在：

C3 到 C6

出题时不能因为某种音程导致明显不同的固定音区，从而泄露答案。

3. 音色

MVP 至少提供两种音色：

- 键盘类音色
- 拨弦类音色

要求：

- Android 和 Windows 听感尽量一致
- 不依赖平台 MIDI 实现
- 不使用来源不明或存在版权问题的音频素材
- 所有音色响度尽量归一
- 不要让某个音程因为音量、持续时间或采样质量不同而泄露答案

可以采用以下方案之一：

1. 运行时生成 PCM/WAV，并缓存生成结果。
2. 使用项目内自带且许可清晰的音频样本。
3. 使用明确支持 Android 和 Windows 的跨平台音频引擎。

如果使用合成音色：

- 键盘类音色可以采用多谐波叠加和指数衰减包络
- 拨弦类音色可以采用简单拨弦合成或类似 Karplus-Strong 的方式
- 请在 UI 中准确描述为“合成键盘”“合成拨弦”，不要冒充真实录制钢琴或吉他

音频频率计算应遵循十二平均律：

$$\text{frequency} = 440 * 2^{((\text{midiNote} - 69) / 12)}$$

建议音频格式：

- 44.1 kHz
- 16-bit
- 单声道
- 带短攻击和释放包络，避免爆音

音频服务必须处理：

- 连续点击重播
- 上一次音频尚未播放结束
- 页面销毁

- App 进入后台
- Windows 窗口关闭
- 避免多个音频实例重叠播放
- 重播时取消旧播放任务

六、MVP 功能

1. 首页

首页包含四个主要入口：

今日练习

自动开始一组的 5 分钟的训练。

第一版建议每组 20 题：

- 5 题热身
- 10 题薄弱音程
- 5 题混合检测

薄弱音程

显示用户最容易混淆的音程组合，例如：

大六度 / 小六度	需要加强
纯四度 / 纯五度	不稳定
大三度 / 小三度	已掌握

点击后直接进入该组合的二选一训练。

自由训练

用户可以配置：

- 参与训练的音程
- 播放方向
- 根音模式
- 音色
- 每组题数
- 音符间隔
- 是否允许重播
- 答案模式

训练报告

展示详细统计。

2. 普通音程识别模式

流程：

1. 生成题目。
2. 播放音程。
3. 用户选择答案。
4. 记录回答。
5. 显示反馈。
6. 进入下一题。

训练页面至少包含：

进度：8 / 20

[播放状态或简洁动画]

[再听一次]

[音程答案按钮]

[不确定]

要求：

- 作答前不要显示键盘或五线谱，避免用户依赖视觉计算
- 作答后可以显示音符位置、半音数和音程名称
- “不确定”必须作为独立回答记录
- 不确定不能算作普通错误猜测
- 重播次数必须单独记录
- 记录从首次播放结束到作答的耗时

3. 二选一混淆训练

这是本 App 的核心功能。

用户选择或系统自动选择两个容易混淆的音程，例如：

- 小六度 / 大六度
- 纯四度 / 纯五度
- 小二度 / 大二度
- 小三度 / 大三度
- 小七度 / 大七度
- 大七度 / 纯八度

页面只显示两个大型答案按钮。

系统应尽量平衡两个答案的出现频率，避免连续大量出现同一个答案。

支持：

- 上行
- 下行
- 和声
- 随机根音
- 音色随机

二选一模式需要单独统计，不要与多选识别模式完全混合。

4. 错题反馈

用户答错后，不要只显示红色错误提示。

反馈区域必须提供：

- 正确答案
- 用户答案
- 两者半音数
- 重听本题
- 播放用户所选音程
- 交替对比播放
- 立即强化一题

例如：

正确答案：大六度
你的答案：小六度

大六度：9 个半音
小六度：8 个半音

按钮：

[重听本题]
[听小六度]
[小六度 ↔ 大六度]
[再练一道]

“交替对比播放” 建议按以下顺序播放：

用户错误答案
正确答案

用户错误答案
正确答案

每次使用相同根音和相同音色，确保用户只比较音程宽度。

错误反馈完成后，系统应在接下来的若干题中增加该混淆对的权重。

七、自适应出题算法

为每位用户维护以下统计：

- 每个音程出现次数
- 每个音程正确次数
- 每个音程错误次数
- 每个音程首次播放正确次数
- 每个音程总重播次数
- 每个音程平均回答时间
- 不确定次数
- 每个“实际音程 → 用户答案”的混淆次数
- 不同方向下的正确率
- 不同根音模式下的正确率
- 不同音色下的正确率

1. 混淆矩阵

例如：

实际：大六度
回答：小六度
累计：12 次

数据结构必须能够表达：

`actualInterval -> selectedInterval -> count`

不要只保存总错误次数。

2. 自适应权重

“今日练习” 建议采用以下权重：

- 50%：当前最弱的混淆对
- 25%：近期答错音程
- 15%：已掌握音程的间隔复习
- 10%：随机检测题

不需要严格保证每组题目完全符合百分比，但总体应接近。

3. 掌握度

掌握度不能只看正确率。

至少考虑：

- 总正确率
- 首次播放正确率
- 重播次数
- 回答时间
- 跨方向稳定性
- 跨根音稳定性
- 跨音色稳定性
- 样本数量

样本过少时，不要显示过高的掌握度。

可以先采用简单、可解释的公式，不需要机器学习。

请把计算逻辑封装在 domain 层，并为其编写测试。

八、训练数据模型

至少设计以下模型。

IntervalQuestion

包含：

- questionId
- correctInterval
- rootMidiNote
- targetMidiNote
- direction
- timbre
- rootMode
- answerOptions
- createdAt

TrainingAttempt

包含：

- attemptId
- sessionId
- questionId

- correctInterval
- selectedInterval, 可为空
- isUncertain
- isCorrect
- replayCount
- responseDuration
- direction
- timbre
- rootMode
- rootMidiNote
- createdAt

TrainingSession

包含:

- sessionId
- trainingMode
- startedAt
- finishedAt
- totalQuestions
- completedQuestions
- correctCount

TrainingConfig

包含:

- enabledIntervals
- direction
- rootMode
- timbreMode
- questionCount
- noteGap
- allowReplay
- answerMode

IntervalStatistics

包含:

- interval
- totalCount
- correctCount
- firstPlayCorrectCount
- replayCount
- uncertainCount
- averageResponseDuration
- masteryScore

九、本地存储

训练数据量不大，优先采用简单、稳定、可迁移的本地方案。

可以采用：

- JSON 文件存储
- 或项目已有的本地数据库方案

如果使用 JSON 文件：

- 文件放在应用支持目录
- 通过 repository 封装
- 加入 schemaVersion
- 写入时尽量使用临时文件加替换，降低文件损坏风险
- 首次启动时自动创建默认数据
- 文件不存在或内容损坏时能够恢复
- 不要因为一条异常记录导致整个 App 无法启动

需要支持：

- 保存设置
- 保存训练记录
- 读取历史统计
- 清空训练数据
- 导出训练数据为 JSON，Windows 和 Android 都应可以操作

导出功能可以放在设置页面。

十、训练报告

报告页面至少展示：

总览

- 总训练题数
- 总正确率
- 首次播放正确率
- 平均重播次数
- 平均回答时间
- 连续训练天数
- 最近 7 天题数

各音程表现

例如：

大六度

总体正确率：82%

首次播放正确率：65%

平均重播：1.8 次

最常误选：小六度

分类表现

按以下维度分别统计：

- 上行
- 下行
- 和声
- 固定根音
- 有限随机根音
- 完全随机根音
- 键盘音色
- 拨弦音色

混淆矩阵

展示实际音程与用户答案之间的混淆情况。

移动端可以使用列表或热度分组，不要求强行放入完整大表格。

Windows 宽屏下可以显示矩阵表格。

十一、设置页面

设置项包括：

- 默认音色
- 默认音符间隔
- 音量
- 是否播放答题反馈音
- 是否自动播放下一题
- 是否显示半音数
- 是否显示英文音程简称
- 清空训练记录
- 导出训练记录
- 关于页面

清空数据必须弹出二次确认。

十二、Windows 端交互

Windows 端增加快捷键：

- Space：重播题目
- 1 到 9：选择对应答案
- 0：选择第十个答案
- U：不确定
- Enter：进入下一题
- Esc：返回或关闭当前反馈面板

要求：

- 页面上适当显示快捷键提示
 - 输入焦点在文本框时不要错误触发训练快捷键
 - 快捷键行为与鼠标点击使用同一业务方法
 - 不要复制两套答题逻辑
-

十三、Android 端交互

Android 端要求：

- 答案按钮有足够大的触摸面积
 - 处理系统返回键
 - 音频播放期间页面退出时立即停止播放
 - 可以使用轻微震动反馈，但必须可关闭
 - 不申请与功能无关的权限
 - MVP 不使用麦克风，因此不要申请录音权限
-

十四、UI 与视觉要求

整体风格：

- 简洁
- 专注
- 不做儿童化设计
- 不使用复杂角色养成
- 不使用体力系统
- 不使用强制排行榜
- 不让动画影响连续训练效率

建议：

- 默认跟随系统浅色/深色模式
- 正确反馈使用明确但不过度刺激的视觉提示
- 错误反馈重点展示对比学习，而不是惩罚

- 桌面端保持合理内容宽度，不要让答案按钮铺满超宽窗口
- 移动端优先单列布局
- Windows 宽屏可以双栏展示题目与统计
- 使用 Material 3
- 所有交互控件提供语义标签
- 重要按钮满足可访问性尺寸要求

首页文案使用中文。

字符串集中管理，为以后国际化保留空间，但本次不需要实现多语言切换。

十五、建议的首版课程

预置以下训练组：

基础音程

- 纯一度
- 纯五度
- 纯八度

大小三度

- 小三度
- 大三度

大小二度

- 小二度
- 大二度

四度与五度

- 纯四度
- 纯五度

大小六度

- 小六度
- 大六度

大小七度

- 小七度
- 大七度

全部音程

包含一个八度内所有音程。

课程预设只是自由训练配置的模板，不要另外实现一套重复的训练引擎。

十六、测试要求

必须为以下逻辑编写单元测试：

1. MIDI 音高与音程半音计算。
2. 上行和下行目标音生成。
3. 根音范围不会越界。
4. 和声音程生成。
5. 两个答案的二选一题目平衡。
6. 自适应权重计算。
7. 混淆矩阵更新。
8. 首次播放正确率计算。
9. 重播次数统计。
10. 掌握度计算。
11. JSON 数据序列化和反序列化。
12. 损坏数据的恢复逻辑。

至少编写以下 Widget 测试：

- 点击重播后 replayCount 增加
- 点击正确答案后进入正确反馈
- 点击错误答案后显示对比播放入口
- 点击“不确定”后正确记录
- 下一题可以正常生成
- Windows 快捷键和按钮调用同一逻辑

保证：

```
flutter analyze  
flutter test
```

能够通过。

如果当前环境具备对应工具链，再执行：

```
flutter build windows  
flutter build apk --debug
```

如果环境缺少 Android SDK、Visual Studio 或其他平台工具链，请明确报告，不能伪造构建成功。

十七、实现顺序

按以下阶段实现。

阶段 1：领域模型和算法

完成：

- 音程定义
- 出题逻辑
- 根音生成
- 自适应权重
- 混淆矩阵
- 统计计算
- 单元测试

阶段 2：音频系统

完成：

- 跨平台音频服务
- 音频生成或资源加载
- 上行、下行、和声播放
- 重播
- 停止播放
- 音频缓存
- 防止重叠播放

阶段 3：训练页面

完成：

- 普通识别
- 二选一训练
- 不确定
- 答题反馈
- 错误对比播放
- 训练进度

阶段 4：本地数据

完成：

- 设置保存
- 训练记录保存
- 统计读取
- 数据恢复
- 清空数据
- JSON 导出

阶段 5：首页和报告

完成：

- 今日练习
- 自由训练
- 薄弱音程
- 报告页面
- 混淆统计

阶段 6：平台体验和测试

完成：

- Windows 快捷键
- Android 触摸体验
- 响应式布局
- 生命周期处理
- analyze
- test
- 可用平台构建验证

十八、本次不实现

以下功能暂不进入 MVP：

- 用户账号
- 云同步
- 排行榜
- 支付
- 广告
- 和弦识别
- 节奏训练
- 旋律听写
- 麦克风模唱检测
- 乐器输入
- 教师端
- 在线课程
- 真实歌曲片段识别
- 推送通知

请为未来扩展保留合理接口，但不要提前实现复杂抽象。

十九、验收标准

完成后应能做到：

1. App 在 Android 和 Windows 上正常启动。
 2. 用户可以选择多个音程进行训练。
 3. 可以播放上行、下行和和声音程。
 4. 可以固定根音或随机根音。
 5. 可以使用至少两种跨平台音色。
 6. 用户可以重播。
 7. 用户可以选择“不确定”。
 8. 答错后可以对比正确音程与错误音程。
 9. 系统能够记录“大六度被误选成小六度”这类具体混淆。
 10. 首页能够显示用户当前最薄弱的混淆组合。
 11. 今日训练会优先安排薄弱组合。
 12. 训练数据在应用重启后仍存在。
 13. Windows 支持键盘快捷键。
 14. 页面在手机和桌面窗口中均可使用。
 15. `flutter analyze` 和 `flutter test` 通过。
-

二十、最终输出要求

完成实现后，请输出：

1. 实现内容摘要。
2. 主要技术决策。
3. 新增或修改的文件列表。
4. 新增依赖及原因。
5. 数据存储位置和数据结构。
6. 音频实现方式。
7. 已执行的测试和构建命令。
8. 测试和构建结果。
9. 当前已知限制。
10. 后续最值得实现的三个功能。

不要只生成 README 或设计文档。请实际创建和修改项目文件，并尽可能运行代码检查与测试。